

AI 智能电商项目 面试问答手册

完成态口径
经典场景题 + 项目落点

详细思路拆解
可直接复述的回答模板

适用岗位：Java 后端 / 全栈开发 / AI 应用开发

使用方法：先遮住回答模板，按照思路口述 2-3 分钟，再对照修正。

使用说明

这不是只针对仓库细节的问答，而是一套面试官无需了解项目也能提出的经典系统设计与场景题。每个答案都采用三层结构：先补充业务背景，再讲通用技术方案，最后落到本项目。回答时不要背完整段落，要用自己的语言说清楚约束、正常路径、异常路径、数据兜底和技术取舍。

通用回答公式

- 1. 场景与目标：先说明要解决什么业务问题。
- 2. 核心约束：一致性、并发、权限、性能或可用性要求是什么。
- 3. 正常方案：按请求顺序讲组件和数据变化。
- 4. 异常方案：覆盖超时、重复、宕机、乱序和补偿。
- 5. 最终兜底：数据库条件、唯一索引、状态机或审计记录。
- 6. 取舍与验证：说明代价，并给出测试或指标。

目录

1. 项目介绍与架构（6题）
2. 前端架构与经典场景（9题）
3. 数据库、订单与库存（8题）
4. 分布式事务、MQ 与可靠性（9题）
5. Redis、缓存与搜索（5题）
6. 身份、权限与 Web 安全（5题）
7. AI、RAG 与安全边界（5题）
8. 测试、性能、部署与复盘（6题）

共 53 道主问题。每道题均包含详细思路和回答模板。

第一章 项目介绍与架构

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q01 请用两分钟介绍这个项目。

思路拆解

- 面试官不知道项目背景，第一句话先定义它是什么，不要从技术栈开始。
- 按业务目标、系统边界、核心链路、个人职责、最大难点、验证结果六步组织。
- 主线只讲商品发布到订单履约，不逐个枚举九个服务。
- 结果必须落到可演示链路、自动化测试和真实指标；没有测量的数据不要编造。

回答模板

这是一个面向消费者和后台运营人员的 AI 智能电商平台。我负责整体架构和核心交易链路，系统由 Vue 商城端、Vue 后台端、Java 微服务和独立 FastAPI AI 服务组成。后端按身份、商品、购物车、订单、库存等业务边界拆分，外部请求统一经过 Gateway。项目最难的是订单与库存位于不同数据库，我通过库存预占、业务唯一键、Outbox、RocketMQ、消费幂等和补偿任务实现最终一致性，并通过并发、重复请求和故障恢复测试验证。AI 只调用受控商品接口，异常时不会影响正常交易。

Q02 为什么选择微服务，而不是单体架构？

思路拆解

- 先承认微服务不是默认答案，小规模业务用模块化单体成本更低。
- 再说明本项目按数据所有权和变化原因划分边界，而不是按表拆服务。
- 列出收益：独立演化、故障隔离、独立扩容；列出代价：调用、部署和一致性复杂度。
- 最后说明克制：核心域做深，搜索、购物车等辅助域保持轻量。

回答模板

如果只是小规模上线，我会优先选择模块化单体。本项目采用微服务，一方面是身份、商品、订单和库存的数据所有权、生命周期和扩展压力不同，另一方面需要验证跨服务一致性和故障隔离。收益是边界清晰、服务可独立部署和扩容；代价是引入网络调用、消息可靠性和运维复杂度。因此我没有把所有功能都做成复杂 DDD，只在订单、库存和商品核心域使用完整领域模型。

Q03 微服务边界是怎么划分的？为什么订单和库存必须分开？

思路拆解

- 使用 DDD 限界上下文解释，而不是回答成 Controller 目录划分。
- 边界判断标准包括业务语言、数据所有权、事务边界、变化频率和扩容特征。
- 订单管理交易承诺和履约状态；库存管理实物数量、预占和流水。

- 强调服务不能直接修改其他服务的表，只能通过内部 API 或事件协作。

回答模板

我按照限界上下文划分服务。订单服务拥有订单、支付、履约和状态历史；库存服务拥有可用量、预占量、已售量和库存流水。两者业务语言和状态机不同，也有独立扩容需求，所以保持独立。订单只保存 reservationNo 等业务引用，不能直接更新库存表；需要即时结果时调用库存内部 API，状态传播则使用事件。这个边界也让订单库存最终一致性成为项目的核心设计。

Q04 从浏览器输入地址到接口返回，完整链路是什么？

思路拆解

- 覆盖 DNS/Nginx、前端静态资源、Axios、Gateway、服务发现、业务服务和数据库。
- 说明 Gateway 做认证与路由，业务服务做最终授权和业务规则。
- 说明 TraceId 在 HTTP、Feign、MQ 和 AI 服务中继续传递。
- 错误最终转换为统一响应，前端根据 HTTP 状态与业务码处理。

回答模板

用户首先通过 Nginx 获取 Vue 静态资源，页面请求由统一 Axios Client 发往 /api。Nginx 转发到 Gateway，Gateway 生成或透传 TraceId、校验 Token 并通过 Nacos 发现目标服务。业务服务再次建立 SecurityContext、执行权限和参数校验，再进入应用层、领域层和仓储层。跨服务即时查询使用 OpenFeign，异步状态传播使用 RocketMQ。响应统一包装为 success、code、message、data、traceId，前端分别处理正常数据、401、403 和业务错误。

Q05 DDD 在项目中的具体体现在哪里？

思路拆解

- 不要只说四层目录，重点是聚合行为和不变量。
- 用 Order、Inventory 两个具体例子说明状态不能由 Controller 随意修改。
- 区分领域事件和跨服务集成事件。
- 说明辅助模块不机械套用复杂 DDD。

回答模板

DDD 主要用于保护核心业务不变量。以订单为例，Order 聚合提供 pay、cancel、ship、confirmReceipt 等行为，并在聚合内部校验合法状态转换；库存预占只能从 RESERVED 转为 RELEASED 或 DEDUCTED。Controller 只负责协议转换，应用服务编排用例，领域层处理规则，基础设施层负责持久化和外部调用。领域事件先描述上下文内部事实，再转换成稳定、版本化的集成事件发布给其他服务。

Q06 如果现在让你重新设计，哪些地方会简化？

思路拆解

- 这是考察反思和架构判断，不要回答项目已经完美。
- 承认初始蓝图偏大，给出模块化单体到微服务的演进路线。
- 保留最有价值的订单库存边界，合并低价值服务。
- 说明简化后的收益和何时再拆分。

回答模板

初始蓝图包含双前端、九个服务、搜索和完整 AI，个人项目范围偏大。重新设计时，我会先使用模块化单体完成身份、商品、会员和系统配置，只独立保留订单、库存与 AI 服务；搜索先使用 MySQL，再根据数据量切换 Elasticsearch。这样能更快形成交易闭环，同时保留订单库存一致性和 AI 隔离两个核心亮点。只有当团队、流量或发布节奏出现独立需求时再继续拆分。

第二章 前端架构与经典场景

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q07 为什么要拆成 mall-web 和 mall-admin 两个前端应用？

思路拆解

- 从用户、权限、交互风格、发布节奏和性能目标解释。
- 商城强调浏览、转化、移动适配；后台强调表格、表单和权限。
- 说明独立构建的代价，以及如何保持接口和规范一致。

回答模板

商城端和后台端面向不同用户。mall-web强调商品浏览、交易转化和移动端体验；mall-admin强调复杂表格、表单、动态菜单和细粒度权限。两者权限模型、依赖和发布节奏不同，因此独立构建部署。代价是公共配置可能重复，我通过统一 HTTP Client 规范、共享接口类型约定和模板化工程配置保持一致，而不是强行共享所有页面组件。

Q08 管理员登录后，前端如何恢复会话并生成动态路由？

思路拆解

- 按时间顺序讲：登录、保存 Token、拉取 bootstrap、写 Store、映射组件、注册路由。
- 页面刷新后 Pinia 内存丢失，需要 restore 和 bootstrap。
- 后端组件标识必须经过前端白名单 registry，不能任意 import。
- 动态路由注册后要重新导航，避免首次匹配进入 404。

回答模板

登录成功后，Auth Store保存短期 Access Token和会话信息，Axios统一注入 Token。前端请求 session bootstrap，一次拿到当前用户、菜单树、权限集合和 permissionVersion，分别写入 Auth、Menu和Permission Store。菜单中的 componentKey 通过前端 component-registry白名单映射成组件，再调用 router.addRoute注册。页面刷新时先恢复 Token，再重新执行 bootstrap；动态路由注册完成后重新导航当前 fullPath，避免首次路由匹配落入404。

Q09 多个请求同时返回 401，如何避免重复刷新 Token？

思路拆解

- 经典并发刷新题。核心是 single-flight：全局只有一个 refresh Promise。
- 其他失败请求等待同一个 Promise，成功后重放原请求。
- 刷新请求本身不能再次触发刷新，否则死循环。
- 失败时一次性清理会话并跳转登录，重放必须限制次数。

回答模板

我在 Axios 层实现 refresh coordinator。第一个401创建唯一的 refresh Promise，后续401不再发起刷新，而是等待同一个 Promise。刷新成功后更新 Token并各自重放原请求；刷新失败则统一清理 Auth、Permission、Menu Store和动态路由，再跳转登录页。刷新接口使用独立 Client或跳过401拦截，并在原请求上增加 retry标记，避免无限循环。

Q10 前端隐藏按钮，为什么后端还要校验权限？

思路拆解

- 区分用户体验与安全边界。
- 浏览器代码和请求都可以被修改，前端权限不可信。
- 路由 meta、v-permission负责展示；Spring Security负责最终授权。
- 补充水平越权：资源归属必须从 Token 身份参与查询。

回答模板

前端权限只改善体验，不能提供安全性。用户可以修改JavaScript、伪造请求或直接调用接口，所以菜单、路由和 v-permission只决定是否展示。后端仍通过Spring Security和权限编码校验每个敏感API；会员资源还要使用Token中的 memberId校验归属，防止查看他人的订单或地址。

Q11 搜索框如何防抖，并避免旧请求覆盖新请求？

思路拆解

- 防抖只减少请求，不能解决响应乱序。
- 使用 AbortController取消旧请求，或使用递增 requestId只接受最新响应。
- 路由 Query作为搜索状态事实来源，便于刷新和分享。
- 处理 Loading、空结果、错误和快速切换筛选条件。

回答模板

输入阶段使用防抖减少无效请求，但响应乱序需要单独处理。每次发起新搜索时通过AbortController取消旧请求，或者记录递增requestId，只允许最新请求更新页面。关键词、分页、排序和筛选条件同步到URL Query，保证刷新和分享可恢复。取消请求不显示错误，真正异常则展示可重试状态。

Q12 商品编辑页为什么不能做成一个巨大组件？

思路拆解

- 从复杂度、校验、复用、性能和测试解释拆分。
- 建议拆为基础信息、分类品牌、属性图片、SKU矩阵、发布校验。
- 统一由父级编辑器维护草稿和提交事务，子组件不要各自保存。

- 说明离开页面未保存提示和后端版本冲突处理。

回答模板

商品编辑包含基础信息、图片、属性和SKU矩阵，如果放在一个组件中，状态耦合、校验和测试都会失控。我把页面拆成多个领域化编辑区，父级ProductEditor维护统一草稿、脏状态和提交动作，子组件只通过明确事件更新局部数据。提交前聚合校验，离开页面时检测未保存修改；后端返回版本冲突时提示用户刷新合并，而不是静默覆盖。

Q13 多规格 SKU 组合如何生成，并保留用户已经填写的价格？

思路拆解

- 规格维度做笛卡尔积，例如颜色乘容量。
- 不能使用数组下标作为身份，应根据排序后的规格键生成稳定 signature/hash。
- 重新生成组合时按 signature 合并旧值。
- 前后端都校验规格唯一性，后端唯一索引兜底。

回答模板

前端对规格维度做笛卡尔积生成SKU组合，并将规格项排序后生成稳定的specificationSignature。规格发生变化时重新生成组合，再按signature合并旧草稿，因此没有变化的SKU可以保留价格、编码和图片。数组下标不能作为SKU身份。提交时前端检查重复和必填，后端再次计算规格Hash并通过唯一索引兜底。

Q14 金额为什么使用分传输？前端如何处理？

思路拆解

- JavaScript Number处理小数存在二进制精度问题。
- 接口以整数分传输；显示时格式化，输入时转换并验证小数位。
- 金额计算由后端负责，前端金额只能作为预览。

回答模板

金额接口统一使用整数分，避免0.1加0.2一类浮点误差。前端只在展示层把299900格式化为2999.00元，表单输入先校验最多两位小数再转成分。最终订单金额由后端根据最新商品价格重新计算，前端和购物车中的金额都不能作为成交依据。

Q15 AI 对话为什么选择 SSE，而不是 WebSocket？

思路拆解

- 先看通信方向：主要是服务器持续向客户端推送生成结果。
- SSE基于HTTP、协议简单、代理友好、支持事件类型和自动重连。
- 需要双向高频实时通信时才更适合WebSocket。

- 说明取消、断线、XSS和结构化事件处理。

回答模板

AI生成主要是请求发出后服务端单向持续推送，SSE已经满足需求，而且基于HTTP，接入Gateway和代理更简单。事件被划分为text.delta、tool.started、tool.completed、structured.result和error，前端按类型更新消息和商品卡片。用户停止生成时使用AbortController取消连接；渲染Markdown时做严格清洗，不能直接使用未经处理的v-html。若未来需要语音或高频双向协作，再考虑WebSocket。

第三章 数据库、订单与库存

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q16 一次创建订单经过哪些步骤？

思路拆解

- 先给主流程，再补本地事务边界和失败路径。
- 订单服务负责协调，但不能直接访问其他服务数据库。
- 价格、商品和地址必须重新校验并保存快照。
- 库存先预占，订单失败后补偿释放。

回答模板

用户提交订单后，订单服务先校验Idempotency-Key，再调用商品服务获取最新可售状态、价格和商品快照，调用会员服务校验地址归属。随后生成orderNo，并用它作为businessNo调用库存服务批量预占。库存成功后，订单服务在本地事务中写订单、订单项、地址快照、状态历史和Outbox。若订单事务失败，则幂等释放库存；购物车清理放在订单成功后，失败只做补偿，不回滚订单。

Q17 如何防止库存超卖？

思路拆解

- 指出先查后改存在并发窗口。
- 数据库条件更新或CAS是最终正确性保障。
- 通过影响行数判断成功，业务唯一键防重复预占。
- Redis锁只能优化竞争，不是最终防线。

回答模板

不能先SELECT再由Java判断，因为多个线程会同时看到相同库存。我在数据库执行带available_quantity大于等于购买量的条件更新，并结合version进行CAS，只有影响一行才算成功。同时在库存预占表上建立businessNo加skuld唯一约束，防止相同订单重复锁定。Redis锁可以减少热点竞争，但库存正确性的最终保障仍然是数据库原子更新和唯一索引。

Q18 一个订单包含多个 SKU，其中一个库存不足怎么办？

思路拆解

- 库存服务拥有所有SKU库存，可在一个本地事务批量处理。
- 按skuld固定顺序更新，降低并发死锁概率。
- 任一失败抛出异常，整个库存事务回滚。
- 若未来库存分片跨库，需要升级为Saga或分片预占补偿。

回答模板

当前库存数据位于同一个库存服务和数据库中，因此批量预占放在一个本地事务内，并按skuld固定顺序执行条件更新，降低死锁概率。任一SKU库存不足就抛出异常，事务整体回滚，不产生部分预占。如果未来按仓库或SKU分片导致跨库，就不能继续依赖单事务，需要记录每个分片结果并使用Saga补偿。

Q19 订单为什么要保存商品和地址快照？

思路拆解

- 订单代表历史交易承诺，不能随着主数据变化。
- 保存名称、规格、成交价、图片和收货信息。
- 快照会有数据冗余，但换来历史可解释性和查询稳定性。

回答模板

订单是下单时形成的交易承诺。商品可能改名、改价或下架，用户也可能修改地址，如果订单每次实时关联主数据，历史订单会发生变化。因此订单项保存商品名称、SKU规格、成交单价和图片快照，订单保存收货地址快照。虽然增加冗余，但保证历史交易可审计，也减少订单详情对其他服务的实时依赖。

Q20 支付与超时取消同时发生，如何避免状态错乱？

思路拆解

- 这是典型状态竞争，不能先读后写。
- 支付和取消都从PENDING_PAYMENT做条件更新，只允许一个成功。
- 延迟消息到达时必须重新查询，而不是直接取消。
- 真实支付中若外部渠道成功但本地已取消，需要退款补偿。

回答模板

支付和取消都只能从PENDING_PAYMENT状态迁移，并通过当前状态加version的条件更新竞争。最终只有一个操作能成功。延迟消息到达时必须重新检查订单状态和deadline，已支付就直接忽略。真实支付场景中，如果渠道已经扣款但本地取消先成功，还需要进入支付对账和原路退款补偿；当前模拟支付则以数据库状态竞争结果为准。

Q21 为什么核心交易表不物理删除？

思路拆解

- 订单、支付、库存流水承担审计、对账、客诉和补偿依据。
- 主数据可以逻辑删除，交易记录不应随意删除。
- 大数据量通过归档、冷热分层和分区解决，而不是直接删除。

回答模板

订单、支付、退款、库存预占和库存流水都是审计与对账证据，也是故障补偿的依据，因此不能物理删除，也不适合普通逻辑删除。商品、角色等主数据可以逻辑删除。交易数据增长后通过归档表、冷热分层、分区和合规保留周期控制成本，而不是在业务请求里直接删除。

Q22 索引怎么设计？联合索引顺序怎么确定？

思路拆解

- 从真实查询和唯一性约束出发，不为每个字段建索引。
- 等值、高选择性、范围和排序结合最左前缀分析。
- 举订单列表和超时扫描两个项目例子。
- 使用EXPLAIN和慢查询验证，考虑写放大和回表。

回答模板

索引首先服务于唯一约束和高频查询。订单列表按memberId、status过滤并按createdAt倒序，可设计memberId、status、createdAt组合索引；超时扫描按status和paymentDeadline查询，则使用status、paymentDeadline。字段顺序结合等值条件、范围条件和排序要求确定。设计后用EXPLAIN、慢查询和实际数据量验证，同时控制索引数量，因为每个索引都会增加写入和存储成本。

Q23 数据库发生死锁怎么办？

思路拆解

- 死锁无法完全消除，重点是减少并正确重试。
- 固定资源访问顺序、缩短事务、避免事务内远程调用。
- 只对可识别且幂等的死锁异常有限重试。
- 保留死锁日志并用业务号追踪。

回答模板

我通过固定SKU更新顺序、缩短事务、建立合适索引以及禁止在数据库事务内执行慢远程调用来降低死锁概率。死锁仍可能发生，数据库会回滚其中一个事务。应用识别死锁异常后，只对具备幂等键的操作进行有限次数、带退避的重试，并记录orderId、skuId和TraceId用于定位，不能对所有异常无限重试。

第四章 分布式事务、MQ 与可靠性

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q24 库存锁定成功，但订单保存失败怎么办？

思路拆解

- 先释放，再补偿；释放必须幂等。
- 补偿任务记录业务号、预占号、重试次数和下次执行时间。
- 超过阈值告警和人工处理，不能无限静默重试。

回答模板

库存锁定成功后如果订单本地事务失败，订单服务立即使用reservationNo调用库存释放。释放接口通过预占状态机保证幂等，已经释放时直接返回成功。如果同步释放也失败，就保存补偿任务，使用指数退避定时重试；超过最大次数进入告警和人工处理列表，并通过业务号核对订单与预占状态。

Q25 调用库存接口超时，如何判断操作是否成功？

思路拆解

- 超时只代表没有收到结果，不代表服务没有执行。
- 写请求必须先生成业务幂等键，并用同一键重试或查询。
- 库存服务已处理时返回原预占结果。
- 不能换一个businessNo重试，否则可能重复扣减。

回答模板

网络超时不能直接当作锁定失败，因为库存事务可能已经提交，只是响应丢失。订单服务使用提交前生成的同一个businessNo重试。库存服务依靠businessNo加skuld唯一约束识别重复请求，已经成功时返回原reservationNo。如果仍无法确认，就进入状态查询和对账任务，在结果明确前不继续创建订单。

Q26 Outbox 解决了什么问题？

思路拆解

- 核心是业务数据库提交和待发送事件之间的原子性。
- 业务记录与Outbox同一本地事务，后台任务投递。
- Outbox不解决重复，发送后宕机会再次投递。
- 说明消费者幂等和积压监控。

回答模板

Outbox解决业务状态已经提交但消息没有可靠记录的问题。订单状态和outbox_event在同一本地事务提交，后台投递器扫描PENDING事件发送MQ，成功后标记SENT。若发送成功但标记前宕机，恢复后会重复发送，所以Outbox保证的是不丢而不是不重复，消费者仍必须幂等。系统还监控PENDING年龄、失败次数和消息积压。

Q27 MQ 消息重复、丢失、乱序分别怎么处理？

思路拆解

- 重复：eventId消费记录加业务状态机。
- 丢失：Outbox重试、MQ持久化、定时扫描/对账兜底。
- 乱序：聚合键顺序队列加状态版本，非法旧事件被拒绝。
- 不要只说MQ配置，要强调业务兜底。

回答模板

重复通过eventId加consumerGroup唯一记录和业务状态机双重幂等；丢失通过Outbox持续投递、Broker持久化、消费重试以及超时订单定时扫描兜底；同一订单或SKU的事件使用orderNo或skuId作为顺序键，并携带版本号，消费者只接受合法的新状态。消息系统提供基础可靠性，最终仍由数据库约束、状态机和对账保证业务正确。

Q28 消息处理到一半服务宕机怎么办？

思路拆解

- 业务更新与消费记录应在同一本地事务。
- 事务提交前宕机则回滚，消息重新投递；提交后ACK前宕机会重复。
- 重复投递由消费幂等返回成功。

回答模板

消费者在一个本地事务中完成业务状态更新和消费记录。如果事务提交前宕机，数据库回滚，消息会重新投递；如果事务已提交但ACK前宕机，消息也会重投，但eventId唯一记录和业务状态机能识别已经完成，直接返回成功。这样不依赖难以保证的Exactly Once，而是实现业务上的一次效果。

Q29 为什么不用 Seata 或两阶段提交？

思路拆解

- 结合业务说明预占模型天然允许中间状态。
- 2PC提高强一致但增加协调器、锁持有和可用性成本。
- 本方案是Saga式补偿和最终一致，代价是开发和对账复杂。

回答模板

订单库存业务天然适合预占再确认，不要求两个数据库在同一时刻绝对一致。两阶段提交会增加协调器依赖、锁持有时间和故障处理复杂度。我选择本地事务加预占、Outbox、幂等和补偿实现最终一致。代价是存在短暂中间状态，需要补偿、对账和监控，但故障边界更清晰，也更适合异步交易流程。

Q30 订单延迟关闭如何实现？延迟消息丢失怎么办？

思路拆解

- 创建订单时保存paymentDeadline，不只依赖消息时间。
- 延迟消息只是触发检查，到期必须重新判断状态。
- 定时扫描超时订单兜底。
- 支付与取消通过CAS竞争。

回答模板

创建订单时固化paymentDeadline并发送RocketMQ延迟消息。消息到达后重新查询订单，只有仍为PENDING_PAYMENT且已超过deadline才执行条件取消，并发布释放库存事件。为防止消息丢失，后台定时扫描status加paymentDeadline索引补偿漏单。支付和取消都通过状态加version的CAS更新竞争，避免已支付订单被误关。

Q31 服务雪崩如何处理？

思路拆解

- 先区分核心与非核心依赖，避免层层无限等待。
- 超时、限流、隔离、熔断、降级和重试必须组合使用。
- 重试只适用于可恢复、幂等请求，并加退避抖动。
- AI和搜索可降级，库存写入不可伪造成功。

回答模板

首先给所有远程调用设置比上游更短的超时，避免线程堆积；按依赖和接口设置并发隔离、限流和熔断。查询类幂等调用可以有限重试并加入退避抖动，写调用不能盲目重试。搜索故障可降级到分类或MySQL基础查询，AI故障降级到普通搜索；库存锁定等核心写操作无法确认时必须失败或进入对账，不能返回虚假成功。

Q32 如何设计分布式链路追踪？

思路拆解

- 入口生成TraceId，HTTP Header、Feign、MQ事件、AI请求全链路传递。
- 日志使用结构化字段，同时保留orderNo、eventId等业务键。
- 指标、日志、追踪分别回答发生了什么、影响多大、路径在哪里。

回答模板

Gateway为没有TraceId的请求生成标识，并注入日志MDC；Feign拦截器继续透传，MQ事件Envelope携带traceId和eventId，AI请求也传递同一标识。所有服务输出结构化日志，并同时记录orderNo、reservationNo等业务键。监控看错误率和延迟，Trace定位跨服务路径，业务日志用于还原状态变化，三者结合排查。

第五章 Redis、缓存与搜索

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q33 缓存穿透、击穿、雪崩分别怎么处理？

思路拆解

- 穿透是查询不存在数据；击穿是单个热点失效；雪崩是大量Key同时失效。
- 分别给出空值/Bloom、互斥重建/逻辑过期、随机TTL/限流降级。
- 强调数据库仍是事实来源。

回答模板

缓存穿透是不断查询不存在的数据，可以使用参数校验、短TTL空值缓存或Bloom Filter；缓存击穿是热点Key瞬间失效，可以使用互斥重建或逻辑过期返回旧值；缓存雪崩是大量Key同时过期或Redis故障，通过TTL随机化、分批预热、限流降级和高可用部署处理。缓存只做加速，数据库仍然是事实来源。

Q34 数据库更新后，如何保证缓存一致性？

思路拆解

- 项目中的商品和配置接受短暂最终一致。
- 使用Cache Aside：更新数据库后删除缓存。
- 删除失败通过重试、事件或版本号修复。
- 订单库存不能把缓存当成正确性判断依据。

回答模板

商品详情和配置使用Cache Aside。写请求先完成数据库事务，再删除缓存，下一次查询回源并重建。删除失败通过重试或配置变更事件再次失效，并设置合理TTL限制脏数据存活时间。对库存和订单等核心数据，缓存只提供查询辅助，最终状态转换和库存扣减仍然以数据库为准。

Q35 Redis 宕机后系统会怎样？

思路拆解

- 按功能区分，不要回答整个系统都不可用。
- 购物车暂时降级；权限缓存和配置缓存回源数据库。
- 核心订单库存正确性不依赖Redis。
- 防止回源流量把数据库压垮。

回答模板

Redis故障会直接影响购物车和部分会话体验，但不会破坏订单库存正确性。权限和配置缓存可以在限流保护下回源数据库，商品详情也可以降级查询MySQL；购物车则提示暂时不可用或使用本地短期状态。恢复后分批预热，避免所有请求同时击穿数据库。系统不会因为Redis异常绕过权限或返回虚假库存。

Q36 Elasticsearch 与商品库不一致怎么办？

思路拆解

- 明确MySQL是事实来源，ES是可重建投影。
- Outbox事件增量同步，eventId与版本号幂等。
- 提供失败重试、单商品修复、全量重建和别名切换。

回答模板

商品MySQL是唯一事实来源，Elasticsearch只是查询投影。商品发布或更新时写Outbox，搜索服务消费事件并按productId和版本号幂等更新。失败进入重试和补偿，可按productId重新同步；严重不一致时从商品库全量构建新索引，校验完成后通过alias原子切换。搜索异常不会反向修改商品数据。

Q37 热点商品成为热 Key 或热点库存行怎么办？

思路拆解

- 区分读热点和写热点。
- 读热点用本地缓存、Redis副本、请求合并；写热点不能只靠缓存。
- 库存写竞争可缩短事务、排队、分桶，但复杂度要与流量匹配。

回答模板

商品详情读热点可以使用本地缓存加Redis两级缓存、请求合并和副本扩展。库存写热点本质是同一SKU行竞争，先通过条件更新、短事务和合理连接池保证正确性；流量继续增长时可以按活动做请求排队、库存分桶或令牌预扣，但需要增加汇总和对账机制。本项目不是秒杀系统，所以不会过早引入复杂分桶。

第六章 身份、权限与 Web 安全

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q38 Access Token 和 Refresh Token 为什么要分开？

思路拆解

- Access Token短期、高频；Refresh Token长期、低频且可撤销。
- Refresh Token持久化时保存摘要并支持轮换。
- 退出、禁用和权限变更的生效策略要说明。

回答模板

Access Token用于高频接口认证，生命周期较短，减少泄露窗口；Refresh Token只用于换取新Access Token，生命周期更长，服务端保存摘要并支持撤销和轮换。退出登录时撤销Refresh Token，管理员禁用时提升tokenVersion或清理会话。若要求Access Token立即失效，可以使用短期黑名单，但需要权衡每次请求查询Redis的成本。

Q39 Gateway 已经认证，业务服务为什么还要鉴权？

思路拆解

- 认证回答是谁，授权回答能做什么。
- Gateway只能做粗粒度入口控制，业务服务理解业务操作和资源归属。
- 内部流量也不能默认可信，防止绕过Gateway。

回答模板

Gateway验证Token并提取可信身份，解决的是用户是谁；能否发布商品、发货或查看某个订单属于业务授权。只有业务服务理解权限编码和资源归属，因此下游仍通过Spring Security和领域规则校验。服务端口不直接暴露公网，内部调用也验证服务身份，避免绕过Gateway访问敏感接口。

Q40 如何防止水平越权和垂直越权？

思路拆解

- 水平越权是访问同权限其他用户资源；垂直越权是低权限执行高权限操作。
- 资源查询必须带可信subjectId；管理操作使用权限编码。
- 不要信任前端传入memberId。

回答模板

水平越权通过资源归属控制，例如订单详情使用orderId加Token中的currentMemberId查询，不能信任前端传入的memberId。垂直越权通过ADMIN主体隔离和product:publish、order:ship等权限编码控制。前端隐藏菜单和按钮只改善体验，后端接口仍必须执行相同权限校验。

Q41 如何防止客户端伪造内部身份 Header?

思路拆解

- 入口删除用户自带内部Header，再注入经过认证的信息。
- 内部接口不向公网路由，并验证服务身份。
- 可用服务Token、请求签名或mTLS，按环境选择。

回答模板

Gateway首先删除客户端传入的X-Subject-Id等内部Header，再根据验证后的Token重新注入可信身份和Traceld。
/api/internal不配置外部路由，业务服务对内部请求继续验证服务Token或签名；生产环境安全要求更高时可使用mTLS。
这样用户即使伪造Header也无法获得内部服务身份。

Q42 前端常见的 XSS、CSRF、CORS 问题如何处理?

思路拆解

- XSS：输出编码、HTML清洗、CSP，尤其AI Markdown。
- CSRF：Cookie认证才是主要风险；SameSite、CSRF Token。
- CORS不是认证机制，只允许可信Origin。
- Token存储方案要结合威胁模型说明。

回答模板

XSS通过Vue默认转义、严格限制v-html、AI Markdown白名单清洗和CSP处理。若Refresh Token使用HttpOnly Cookie，需要SameSite、Secure和CSRF Token防止跨站请求；若Token放Web存储，则更需要控制XSS风险。CORS只限制浏览器跨域读取，不是身份认证，因此Gateway只允许配置中的可信Origin，后端权限校验不能依赖CORS。

第七章 AI、RAG 与安全边界

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q43 AI 如何保证不编造商品、价格和库存？

思路拆解

- 模型只负责意图解析和解释，不是事实来源。
- Tool Calling查询真实候选，返回前进行结构化校验。
- 商品ID、价格、库存由Java业务服务最终确认。

回答模板

AI先把自然语言解析成品类、预算和使用场景，再调用白名单search_products工具获取真实候选。模型只基于候选生成推荐理由，最终返回前由工具层或Java服务重新校验商品ID、上下架状态、价格和库存，并使用Schema约束结构化输出。没有真实候选时明确返回无匹配，不能让模型补造商品。

Q44 为什么 AI 服务不能直接访问业务数据库？

思路拆解

- 数据所有权、权限、审计和模型演进都要求经过业务服务。
- AI只能使用最小权限的受控工具。
- 写操作要二次确认、幂等和业务服务校验。

回答模板

商品和订单的数据所有权属于Java业务服务。AI直连数据库会绕过权限、状态机、审计和服务边界，也会与表结构强耦合。因此AI只调用受控内部API，工具参数和结果都有Schema。涉及取消订单等写操作时，必须验证当前会员、二次确认并携带幂等键，最终规则仍由订单服务执行。

Q45 RAG 的完整流程是什么？如何评估效果？

思路拆解

- 上传、解析、切分、向量化、入库、检索、重排、生成、引用。
- 知识文档是事实来源，向量索引可重建。
- 评估检索Recall@K、答案正确性、忠实度、引用命中和延迟成本。

回答模板

后台上传知识文档后保存原文件到MinIO，异步解析和切分，生成Embedding写入向量库，并记录版本和处理状态。问答时先进行权限和问题改写，再向量检索Top-K，必要时关键词混合检索和重排，最后要求模型只根据上下文回答并返回引用。离线评估检索Recall@K、答案正确性和忠实度，线上监控无答案率、延迟、Token成本和用户反馈。

Q46 如何防止 Prompt Injection 调用危险工具？

思路拆解

- 用户文本永远是不可信数据，不能改变系统权限。
- 工具白名单、参数Schema、最小权限和服务端授权。
- 读写工具隔离，写操作二次确认。

回答模板

系统不会因为用户说忽略规则就开放工具。Agent只能看到白名单工具，参数必须通过Schema校验，每次调用携带真实用户身份并由Java服务重新授权。读取商品和写订单工具分离，写操作必须显式二次确认、幂等和审计。模型输出不能直接拼SQL、URL或组件代码执行，工具结果也按不可信外部数据处理。

Q47 AI 服务不可用时系统如何降级？

思路拆解

- AI不在核心交易链路，故障隔离是架构前提。
- 设置超时、并发隔离、熔断和友好降级结果。
- 前端保留普通搜索入口，不能无限重试。

回答模板

AI是增强能力，不参与商品下单和支付事务。Java调用AI时设置严格超时和并发隔离，连续失败后熔断；前端显示暂时不可用并提供普通搜索入口。已生成的商品卡片仍由商品服务验证。系统不会因为AI超时阻塞线程或影响购物车、订单和库存服务。

第八章 测试、性能、部署与复盘

回答建议：先用一句话给结论，再展开机制；面试官追问时再进入异常路径、取舍和验证。

Q48 这个项目怎么测试？

思路拆解

- 按测试金字塔回答：领域单测、应用集成、契约、E2E、性能和故障测试。
- 列出项目最关键而不是最多的场景。
- 外部依赖尽量使用Testcontainers或稳定替身。

回答模板

领域层重点单测订单状态机、库存预占和金额计算；应用层集成测试数据库事务、唯一约束和安全配置；服务间用契约测试保证DTO和错误码兼容；E2E覆盖商品发布、下单、支付、发货、收货。专项测试重复提交、并发超卖、重复消息、延迟关单和补偿恢复。前端测试Token并发刷新、动态路由、权限指令、SKU编辑器和关键页面交互。

Q49 如何验证系统确实不会超卖？

思路拆解

- 设计库存小于并发请求数的竞争测试。
- 不仅看HTTP成功数，还要核对数据库不变量和预占唯一性。
- 多轮、不同并发和失败注入。

回答模板

我准备固定100件库存，让200个并发请求使用不同订单号竞争同一SKU。测试结束后核对成功预占数必须等于100，available为0，reserved加sold与成功业务一致，不能出现负数或同一业务重复预占。测试重复执行，并加入超时重试和取消释放，最终再次核对库存守恒，而不是只看接口返回。

Q50 压测指标怎么看？发现瓶颈后怎么定位？

思路拆解

- 核心指标：吞吐、P50/P95/P99、错误率、资源和连接池。
- 按浏览器/网关/服务/数据库/MQ链路逐层定位。
- 必须填真实测量数据，不能凭空给TPS。

回答模板

我同时观察TPS、P50/P95/P99、错误率、CPU、GC、线程池、数据库连接池、慢SQL和MQ积压。先用Trace定位耗时最大的调用，再结合应用指标和数据库执行计划判断是锁竞争、连接不足还是慢查询。优化后使用相同数据和压测模型复测，并报告真实前后对比。项目介绍中的并发数和延迟只使用保存过报告的结果。

Q51 如何做到一键部署和环境隔离？

思路拆解

- 镜像化前后端和服务，Compose编排本地演示。
- 配置和Secret通过环境变量或Secret管理，不写入仓库。
- Flyway负责数据库版本，健康检查控制启动依赖。
- 测试、开发和生产配置隔离。

回答模板

各应用生成独立镜像，本地使用Docker Compose启动MySQL、Redis、Nacos、RocketMQ、Elasticsearch、MinIO以及应用服务。数据库由各服务Flyway自动迁移，Compose使用健康检查和依赖条件控制启动。环境差异通过环境变量和Nacos技术配置注入，Secret不提交Git；前端构建时注入API地址，生产由Nginx统一反向代理。

Q52 项目最大的不足是什么？

思路拆解

- 承认范围和复杂度问题，但同时给出改进路线。
- 不要把缺点包装成优点，也不要否定整个项目。
- 可从容量、部署复杂度、AI评估和生产级安全谈。

回答模板

项目最大的问题是初始范围偏大，微服务数量相对个人项目流量存在过度设计，也没有真实支付和生产规模数据。我的改进是优先保证订单库存闭环和故障测试，将搜索、RAG和监控作为独立迭代；架构上评估合并身份与系统等低负载服务。生产化还需要完善密钥管理、多机容灾、真实支付对账、容量规划和持续AI评估。

Q53 如果线上出现订单和库存不一致，你怎么排查？

思路拆解

- 先止损再定位：限制异常写入、保留现场。
- 使用orderNo、reservationNo、eventId和TraceId串联。
- 核对订单状态历史、库存预占、流水、Outbox和消费记录。
- 修复必须幂等、可审计，并补充自动化规则。

回答模板

首先确认影响范围，必要时暂停相关SKU交易，避免继续扩大。然后用orderNo和reservationNo核对订单状态历史、库存预占与库存流水，再用eventId和TraceId检查Outbox、MQ投递和消费记录，判断是业务事务失败、事件未发送、消费失败还是人工调整导致。根据事实执行幂等补偿并记录审计，之后增加对账规则、告警和回归测试，防止同类问题再次发生。

附录 A 90 秒项目介绍模板

这是一个面向消费者和后台运营人员的 AI 智能电商平台。我负责整体架构和核心交易链路。系统包含 Vue 商城端、Vue 后台端、Java 微服务和独立 FastAPI AI 服务，后端按照身份、商品、购物车、订单和库存等限界上下文拆分，外部请求统一经过 Gateway。项目的主要难点是订单与库存位于不同数据库，无法依赖单机事务。我采用库存预占、数据库条件更新和业务唯一键解决超卖与重复锁定，并通过 Outbox、RocketMQ、消费幂等和补偿任务实现最终一致性。前端实现了双 Token 登录、并发刷新协调、动态菜单、动态路由和按钮权限；AI 只通过白名单工具查询真实商品，故障时降级到普通搜索，不影响交易。最后通过商品发布到订单履约的端到端测试、库存并发测试、重复消息测试和故障恢复测试验证核心链路。

附录 B 回答自检清单

- ☐ 我是否先让不了解项目的面试官听懂业务场景？
- ☐ 我是否说明了数据由哪个服务拥有？
- ☐ 我是否讲清正常路径以及至少一个失败路径？
- ☐ 我是否用数据库条件、唯一索引或状态机作为最终兜底？
- ☐ 我是否区分了前端体验控制和后端安全边界？
- ☐ 我是否区分了同步调用和异步事件？
- ☐ 我是否承认方案代价，而不是把所有技术都说成优点？
- ☐ 我引用的吞吐、延迟和并发数据是否来自真实报告？
- ☐ 面试官要求看代码时，我能否迅速定位对应模块？

练习节奏：第一遍只看思路；第二遍限时口述；第三遍让同学随机追问；第四遍脱离文档画出架构、下单、支付和超时关单四张图。